Podręcznik: podstawy tworzenia frontendu aplikacji (przegląd)

Strona:	LeIA 2026	Wydrukowane przez użytkownika:	Tymon Kwiatkowski
Kurs:	Zaawansowane aplikacje internetowe (2026Z)	Data:	piątek, 2 października 2026, 12:07
Książka:	Podręcznik: podstawy tworzenia frontendu aplikacji (przegląd)		

Opis

Spis treści

1. Strona tytułowa

2. Front-end - wprowadzenie

3. HTML5 - język opisu struktury
3.1. Podstawowa struktura dokumentu HTML5
3.2. Wybrane znaczniki semantyczne
3.3. Formularze
3.4. Element Canvas

4. CSS3 - język opisu wyglądu
4.1. Podstawowe selektory
4.2. Selekcja oparta na relacji
4.3. Pseudoklasy
4.4. Pseudoelementy
4.5. Selektory atrybutów
4.6. Filtry

5. Multimedia w aplikacji internetowej
5.1. Osadzanie filmów i nagrań audio
5.2. Formaty plików graficznych

6. Język JavaScript
6.1. Biblioteka jQuery
6.2. React

7. Responsive Web Design
7.1. Stopniowe ulepszanie strony

8. Dostępność cyfrowa
8.1. Zalecenia WCAG
8.2. Kluczowe zasady WCAG - "POUR"
8.3. Jak uzyskać zgodność?
8.4. Zgodna wersja alternatywna
8.5. Poziom wsparcia technologii wspomagających
8.6. Kryteria sukcesu
8.7. Atrybuty WAI-ARIA
8.8. Walidatory
8.9. Przykładowe wdrożenia

1. Strona tytułowa

Niniejszy moduł omawia wybrane zagadnienia i narzędzia wykorzystywane przy projektowaniu i implementacji frontendowej części aplikacji internetowych.

Słowa kluczowe: aplikacje internetowe, programowanie, *frontend*, klient, React, Angular, Vue, framework, kompresja, narzędzia deweloperskie

Wymagana wiedza bazowa: podstawy programowania aplikacji internetowych, podstawowa znajomość języka JavaScript oraz HTML i CSS

Autor: dr inż. Przemysław Korpas, Instytut Radioelektroniki i Technik Multimedialnych

2. Front-end - wprowadzenie

Istotnym elementem aplikacji internetowej jest jej interfejs użytkownika, gdyż to z nim użytkownik bezpośrednio wchodzi w interakcję. Tym samym interfejs ten jest postrzegany i oceniany w pierwszej kolejności subiektywnie, na podstawie szeregu zaznanych wrażeń, obserwacji, itp. Przekłada to się na pewien wachlarz cech aplikacji, które łączą się w dość szerokie pojęcie określane z języka angielskiego jako tzw. *User eXperience* (UX). Na ten składa się zarówno warstwa artystyczna w postaci projektu graficznego interfejsu użytkownika, jak i jego cechy szczegółowe wynikające bezpośrednio ze sposobu jego oprogramowania.

Niniejsza część podręcznika stawia sobie za cel przeprowadzenia Czytelnika przez najważniejsze (zdaniem autora) zagadnienia techniczne związane z projektowaniem interfejsu użytkownika aplikacji internetowej, odsyłając do źródeł po rozwinięcie i szczegóły omawianych tutaj zagadnień. Na warstwie artystycznej nie będziemy się specjalnie skupiać, za to weźmiemy na warsztat różnego rodzaju technikalia, próbując odpowiedzieć na następujące pytania:

1. W jaki sposób poprawnie ustrukturyzować treść prezentowaną na stronie internetowej?
2. W jaki sposób można wpływać na wygląd tych treści?
3. W jaki sposób wykorzystać multimedia?
4. Do czego potrzebuję JavaScriptu na front-endzie?
5. Jak sprawić, aby aplikacja była poprawnie wyświetlana na urządzeniach różnego typu? Słów kilka o Responsive Web Design.
6. Jak sprawić, aby aplikacja spełniała wymagania w zakresie dostępności cyfrowej?
7. Jakie narzędzia pomocnicze oferuje współczesna przeglądarka internetowa?

3. HTML5 - język opisu struktury

Terminem HTML5 (ang. *HyperText Markup Language*) określa się aktualną wersję języka znaczników HTML, który jest obecnie podstawowym standardem używanym do tworzenia i strukturalizacji treści na stronach i w aplikacjach internetowych. Jest to rozwinięcie wcześniejszych wersji HTML, które wprowadza nowe funkcje i elementy. Zakłada się przy tym, że specyfikacja tego standardu podlega i będzie podlegać sukcesywnym zmianom, dlatego pod pojęciem HTML5 rozumiemy tzw. żywy standard, który ciągle ewoluuje. Aktualną specyfikację języka HTML5 znajdziesz na stronie HTML Living Standard.

Zakładam, że Czytelnik niniejszego podręcznik ma już opanowane podstawy języka HTML, zatem w tym rozdziale skupię się wyłącznie na tych elementach składni, które są nowe i niezbędne dla zapewnienia właściwej struktury dokumentu na potrzeby implementacji responsywności aplikacji oraz zapewnienia dostępności cyfrowej, bądź też oferują ciekawe możliwości:

- znaczniki semantyczne,
- wsparcie dla multimediów,
- formularze i nowe typy pól wejściowych,
- obsługa grafiki i animacji.

Lektura uzupełniająca: MDN Learn Web Deveopment - Structuring content with HTML

3.1. Podstawowa struktura dokumentu HTML5

Podstawowa struktura dokumentu zapisanego w języku HTML5 wygląda następująco:

```
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Opis strony">
    <title>Strona w HTML5</title>
</head>
<body>
    <header>
        <h1>Witamy na naszej stronie</h1>
        <nav>
            <ul>
                <li><a href="#o-nas">O nas</a></li>
                <li><a href="#kontakt">Kontakt</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <section id="o-nas">
            <h2>O nas</h2>
            <p>Tworzymy nowoczesne strony internetowe w HTML5.</p>
        </section>
        <section id="kontakt">
            <h2>Kontakt</h2>
            <form>
                <label for="email">Twój e-mail:</label>
                <input type="email" id="email" name="email" required>
                <button type="submit">Wyślij</button>
            </form>
        </section>
    </main>
    <footer>
        <p>&copy; 2024 Wszystkie prawa zastrzeżone</p>
    </footer>
</body>
</html>
```

Powyższy plik można pobrać tutaj.

Dobrym pomysłem na rozpoczęcie nowego projektu może być także skorzystanie z gotowego szablonu "HTML5 Boilerplate" definiującego strukturę dokumentu HTML5 oraz podstawowe style CSS. W ramach tego projektu dostępna jest także wyczerpująca dokumentacja opisująca poszczególne pliki i ich zawartość.

3.2. Wybrane znaczniki semantyczne

Język HTML5 wprowadza nowe znaczniki semantyczne, które lepiej definiują strukturę strony internetowej. Dzięki nim kod jest bardziej czytelny zarówno dla programistów, jak i dla przeglądarek oraz narzędzi wspomagających (np. czytników ekranowych).

Element <header>

Może pełnić rolę nagłówka strony lub sekcji. Liczba tych elementów nie jest ograniczona, np. każdy element <section> może posiadać swój nagłówek. Więcej przeczytasz tutaj.

Element <footer>

Może pełnić rolę stopki strony lub sekcji. Liczba tych elementów nie jest ograniczona, np. każdy element <section> może posiadać swoją stopkę. Więcej przeczytasz tutaj.

Elementy <article>

Element <article> reprezentuje samodzielną, niezależną treść, która może być czytana i rozumiana w oderwaniu od reszty strony. Treść wewnątrz <article> powinna mieć znaczenie również poza kontekstem strony, na której się znajduje.

Element <section>

Element <section> służy do organizowania treści w sekcje tematyczne. Sekcje są częścią większej całości i zazwyczaj współdziałają z innymi częściami strony, tworząc jej strukturę. W odróżnieniu od <article>, zawartość <section> nie musi być samodzielna ani niezależna.

Przykład

Powyższe elementy języka HTML5 można wykorzystać do zorganizowania treści w następujący sposób:

```
<article>
    <header>
        <h2>Jak używać HTML5?</h2>
        <p>Autor: Jan Kowalski</p>
    </header>
    <section>
        <h3>Wprowadzenie</h3>
        <p>HTML5 to nowoczesny standard tworzenia stron internetowych...</p>
    </section>
    <section>
        <h3>Nowe funkcje HTML5</h3>
        <p>HTML5 wprowadza takie elementy, jak <article>, <section>...</p>
    </section>
    <footer>
        <p>Opublikowano 1 stycznia 2024</p>
    </footer>
</article>
```

Element <nav>

Element <nav> jest używany do definiowania obszaru nawigacyjnego. Służy do grupowania linków, które umożliwiają użytkownikom przemieszczanie się między różnymi częściami tej samej strony, innymi stronami w witrynie lub nawet zewnętrznymi zasobami. Element <nav> powinien być używany tylko wtedy, gdy zawiera kluczowe linki nawigacyjne. Oznacza to, że nie należy używać elementu <nav> do grupowania wszystkich linków na stronie (np. pojedynczych odnośników w treści artykułu), ponieważ nie byłyby to główne elementy nawigacyjne.
Użycie elementu <nav> jest lepsze pod względem semantycznym do budowy menu niż tradycyjne obudowanie grupy odnośników <a> w listę nienumerowaną <ul>. Więcej przeczytasz tutaj.

Element <aside>

Element <aside> służy do przedstawiania treści, które są dodatkowe lub poboczne w stosunku do głównej treści dokumentu. Jest to semantyczny znacznik, który oznacza, że zawarte w nim informacje mają charakter uzupełniający, ale nie są bezpośrednio związane z główną treścią.
Przykłady zastosowania:

- jako pasek boczny strony,

- jako przypis lub notatka wewnątrz sekcji,
- dla reklamy lub treści promocyjnych.

Więcej przeczytasz tutaj.

Element <main>

Element <main> w języku HTML5 jest semantycznym znacznikiem używanym do oznaczania głównej treści strony. Zawiera najważniejsze informacje związane z celem strony lub aplikacji. W związku z tym nie mogą znaleźć się w jego wnętrzu elementy, które są powtarzalne na wielu stronach, takie jak nagłówek (<header>), stopka (<footer>) czy paski nawigacyjne (<nav>). Więcej przeczytasz tutaj.

Specyfikacja języka

3.3. Formularze

Język HTML5 wprowadza nowe atrybuty i typy pól formularzy, które ułatwiają tworzenie bardziej funkcjonalnych formularzy bez potrzeby stosowania JavaScript.

Adres email

Gdy ten typ jest używany, wprowadzona wartość musi być adresem e-mail, aby była prawidłowa. Każda inna zawartość powoduje, że przeglądarka wyświetla błąd po przesłaniu formularza.

```
1.  <input <span class="" style="color: rgb(0, 61, 245);">type="email"</span> id="email" name="email" />
```

Możliwa jest także walidacja poprawności wprowadzonego adresu za pomocą atrybutu "pattern".

Pole wyszukiwania

Ten typ znajduje zastosowanie w polach wyszukiwania. Przeglądarki takie pola zwykle rysują w nieco inny sposób. Ponadto wartości wpisywane w pole wyszukiwania mogą być później wykorzystane przez przeglądarkę do celów automatycznego uzupełniania podobnych zapytań.

```
1.  <input type="search" id="search" name="search" />
```

Numer telefonu

Ten typ pola na urządzeniach mobilnych jest przydatny we wszystkich sytuacjach, gdy celowe jest wprowadzenie wartości liczbowej za pomocą klawiatury numerycznej. Oznacza to, że wprowadzać do takiego pola można dowolne wartości, niekoniecznie numery telefonu.

```
1.  <input type="tel" id="tel" name="tel" />
```

Adres URL

Pole służące do wprowadzania adresu URL. Na dynamicznej klawiaturze mogą zostać wyświetlone dodatkowe przyciski, np. dwukropek, kropka, slash itp. Obowiązuje też prosta walidacja.

```
1.  <input type="url" id="url" name="url" />
```

Pole numeryczne

Ta kontrolka wygląda jak pole tekstowe, ale pozwala tylko na wprowadzanie liczby i zwykle udostępnia przyciski w postaci spinnera do zwiększania i zmniejszania wartości kontrolki. Na urządzeniach z dynamicznymi klawiaturami zwykle wyświetlana jest klawiatura numeryczna. Za pomocą atrybutów min, max, step można kontrolować zakres wprowadzanych wartości oraz ich rozdzielczość.

```
1.  <pre><code><input type="number" name="change" id="pennies" min="0" max="1" step="0.01" />
```

Suwak

Innym sposobem na wprowadzanie wartości numerycznej jest użycie kontrolki w formie suwaka. Konfiguracja analogiczna jak powyżej.

```
1.  <pre><code><input type="range" name="change" id="pennies" min="0" max="1" step="0.01" />
```

Kolor

Kolor może być reprezentowany na różne sposoby: poprzez składowe RGB, składowe HSL, opis słowny itp. W przypadku kontrolki pozwalającej wybrać kolor, przeglądarka wyświetla przycisk, po kliknięciu którego przeglądarka wyświetla domyślne okno systemu operacyjnego pozwalającego dokonać wyboru koloru. Po zatwierdzeniu wyboru, zwracana jest charakterystyczna dla języka HTML 6-znakowa wartość szesnastkowa opisująca wartości składowych RGB koloru.

```
1.  <input type="color" name="color" id="color" />
```

Wybór daty

Mamy tutaj kilka możliwości. Począwszy od podstawowego pola wyboru daty:

```
1.  <input type="datetime-local" name="datetime" id="datetime" />
```

poprzez typy pozwalające wprowadzić konkretnie:

- czas - type="time",
- dzień tygodnia - type="week",
- miesiąc - type="month",

a skończywszy na wprowadzeniu ograniczeń co do zakresu wprowadzanych dat jak i granulacji:

```
<label for="myDate">Kiedy chcesz jechać na urlop w nadchodzące wakacje?</label><br />
<input
  type="date"
  name="myDate"
  min="2025-06-01"
  max="2025-08-31"
  step="7"
  id="myDate" />
```

Opracowano na podstawie https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/HTML5_input_types.

3.4. Element Canvas

Element <canvas> pozwala na dynamiczne rysowanie grafiki, tworzenie animacji, gier, wizualizacji danych, a nawet interaktywnych aplikacji bez potrzeby korzystania z zewnętrznych wtyczek. Korzystając z języka JavaScript możliwe jest
Za pomocą JavaScriptu możemy rysować na obszarze płótna, który reprezentuje element <canvas>.

Element <canvas> sam w sobie jest tylko kontenerem na grafikę, czyli tzw. płótnem, na którym odbywa się rysowanie. Aby coś na nim narysować, musisz użyć JavaScriptu. Podstawowe atrybuty elementu <canvas> to:

- width - definiujący szerokość obiektu wyrażoną w pikselach,
- height - definiujący wysokość obiektu wyrażoną w pikselach.

Aby coś narysować na płótnie, potrzebujesz kontekstu graficznego – obiektu, który dostarcza metod do rysowania (np. linii, kształtów, obrazów). Dostępnych jest kilka różnych kontekstów, które znacząco zmieniają sposób wykorzystania płótna:

- 2d - tworzy obiekt reprezentujący kontekst dwuwymiarowy (wybór sugerowany na początek),
- webgl, webgl2 - tworzy obiekt reprezentujący kontekst trójwymiarowy (dostępne tylko w przeglądarkach implementujących WebGL API odpowiednio w wersji 1 lub 2),
- webgpu - tworzy obiekt reprezentujący kontekst trójwymiarowy do renderowania grafiki za pomocą WebGPU (dostępne tylko w przeglądarkach implementujących WebGPU API),
- bitmaprenderer - tworzy prosty kontekst pozwalający na podmianę treści płótna przy użyciu ImageBitmap.

Przykład

Prosty przykład rysujący prostokąt na płótnie.

```
<canvas id="myCanvas" width="500" height="300" style="border:1px solid #000;"></canvas>

<script>
    // Pobierz element <canvas>
    const canvas = document.getElementById('myCanvas');

    // Uzyskaj kontekst 2D
    const ctx = canvas.getContext('2d');

    // Rysuj prostokąt
    ctx.fillStyle = 'blue'; // Ustaw kolor wypełnienia
    ctx.fillRect(50, 50, 200, 100); // Rysuj prostokąt (x, y, szerokość, wysokość)
</script>
```

Gdzie się plasuje Canvas względem innych technik rysowania w przeglądarkach internetowych?

Istnieją cztery sposoby rysowania rzeczy w przeglądarce: Canvas, SVG, CSS i bezpośrednia animacja DOM. Canvas różni się od pozostałych trzech:

- **SVG:** SVG to wektorowy sposób rysowania grafiki. Każdy kształt ma obiekt, do którego można dołączyć obsługę zdarzeń. Przy powiększeniu krawędzie kształtu pozostają gładkie, podczas gdy Canvas stałby się pikselowany.
- **CSS:** CSS to tak naprawdę stylizacja elementów DOM (ang. *Document Object Model*). Ponieważ nie ma obiektów DOM dla rzeczy rysowanych w Canvas, nie można użyć CSS do ich stylizacji. CSS wpłynie tylko na prostokątny obszar samego Canvas, więc możesz ustawić obramowanie i kolor tła, ale to wszystko.
- **Animacja DOM:** DOM definiuje obiekty dla wszystkiego na ekranie. Animacja DOM, za pomocą CSS lub JavaScript do przesuwania obiektów, może być w niektórych przypadkach płynniejsza niż w przypadku Canvas, ale zależy to od implementacji przeglądarki.

Do czego w praktyce mogę zastosować Canvas?

Element Canvas daje potężne możliwości w sytuacjach, gdy niezbędne jest przygotowanie wizualizacji, których złożenie z elementów oferowanych przez język HTML byłoby bardzo złożone (pod względem nakładu pracy potrzebnego do zakodowania dane funkcjonalności) lub kosztowne (nieefektywne pod względem szybkości działania). Poniżej zamieszczono odnośniki do kilku przykładów zastosowań elementu Canvas.

Przykład

Tworzenie wykresów 2D: Josh Marinacci "HTML Canvas Deep Dive" / Hands On: Making Charts

Przykład

Stworzenie prostej gry 2D: Josh Marinacci "HTML Canvas Deep Dive" / Making a Game

Przykład

Grafika 3D z wykorzystaniem WebGL oraz ThreeJS:

- Google Earth - popularna wizualizacja naszej planety,
- The Boat - cyfrowa kreacja, która umiejętnie łącząc wizualizację, tekst i dźwięk, wciąga użytkowników w intrygującą narrację,
- BioDigital Human - prezentuje misternie szczegółowy i interaktywny model 3D ludzkiego ciała, oferując wciągającą podróż przez złożone układy organizmu,
- Particle Love - płynna, interaktywna symulacja ruchu cząstek.

Przykład

Praktyczny przykład, z którym swego czasu mierzył się autor niniejszego podręcznika. Otóż celem było prezentowanie na bieżąco wizualizacji widma sygnału przetwarzanego w czasie rzeczywistym. Nie wchodząc w szczegóły matematyczne: co najmniej **kilkanaście razy na sekundę** chcieliśmy rysować wykres typu y=f(x) **składający się z kilku do kilkudziesięciu tysięcy punktów**. Okazuje się, że przy tak postawionych wymaganiach wszystkie przetestowane biblioteki do rysowania wykresów w jakimś aspekcie były dyskwalifikowane - a to ze względu na zbyt długi czas rysowania, a to przez brak możliwości odpowiedniego skonfigurowania wyglądu wykresu do potrzeb projektowanej aplikacji... Finalnie skończyło się na tym, że zaimplementowana została własna biblioteka rysująca wykres i posiadająca wszystkie potrzebne funkcjonalności, a początkiem tego rozwiązania był proof-of-concept szybkiego rysowania wykresu właśnie za pomocą elementu Canvas. Ten POC możesz uruchomić klikając tutaj. Zapraszam do przeanalizowania jego kodu źródłowego (skrót klawiaturowy Ctrl+U w przeglądarce). Przykład demonstruje, że przy odpowiedniej obróbce wstępnej danych możliwe jest rysowanie wykresu z prędkością 60 klatek na sekundę, co spełnia postawione wymagania z dużym zapasem.

Canvas a dostępność cyfrowa

Należy jednak mieć na uwadze, że treści prezentowane w elemencie Canvas są niewidoczne np. dla czytników ekranów wykorzystywanych przez osoby niewidome, co prowadzi do pogorszenia dostępności cyfrowej takiej strony. Niezbędne są wtedy dodatkowe zabiegi, np. w postaci zapewnienia opisów alternatywnych prezentowanych wizualizacji.

Literatura

Z uwagi na obszerność tematu, osoby zainteresowane odsyłam do ciekawych opracowań zewnętrznych:

- MDN Web Docs - Canvas element,
- MDN Web Docs - Canvas tutorial,
- Josh Marinacci "HTML Canvas Deep Dive" (nie jest to pozycja pierwszej młodości, ale wciąż przydatna),
- HTML5 Canvas Cheat Sheet.

4. CSS3 - język opisu wyglądu

Język CSS3 (ang. *Cascading Style Sheets Level 3*) to język arkuszy stylów, który służy do opisywania wyglądu i formatowania elementów na stronie internetowej w języku HTML. CSS3 pełni kluczową rolę w oddzieleniu warstwy prezentacyjnej (wyglądu) od warstwy strukturalnej (treści) strony, co pozwala na łatwiejsze zarządzanie i większą elastyczność w projektowaniu stron internetowych.

4.1. Podstawowe selektory

Selektory w języku CSS3 to mechanizmy służące do określania, które elementy HTML mają zostać sformatowane za pomocą reguł CSS. Są kluczowym elementem CSS, ponieważ pozwalają wybrać elementy na stronie i zastosować do nich określone style (np. kolory, marginesy, rozmiar czcionki).

W CSS3 wprowadzono kilka nowych selektorów, które rozszerzają możliwości stylizowania elementów HTML w jeszcze bardziej precyzyjny i złożony sposób.

Podstawowe selektory

Selektor uniwersalny (*)

- Pasuje do wszystkich elementów na stronie.
- Jest użyteczny, gdy chcesz zastosować styl do całej strony lub do wszystkich elementów.

Przykład

```
* {
    margin: 0;
    padding: 0;
}
```

Wyzeruje marginesy i paddingi dla wszystkich elementów na stronie.

Selektor typu (np. h1, p)

- Styluje wszystkie elementy danego typu (tagu) HTML, np. wszystkie nagłówki <h1> lub akapity <p>.

Przykład

```
h1 {
    color: blue;
    font-size: 24px;
}
```

Ustawia kolor tekstu i rozmiar wszystkich nagłówków <h1>.

Selektor klasy (.classname)

- Wybiera wszystkie elementy, które mają przypisaną określoną klasę za pomocą atrybutu class.

Przykład

HTML:

```
<div class="box">
    To jest ramka
</div>
<div class="box">
    To też jest ramka
</div>
```

CSS:

```
.box {
    border: 2px solid black;
    padding: 10px;
}
```

Zastosuje styl do wszystkich elementów o klasie box.

Selektor identyfikatora (#id)

- Wybiera jeden unikalny element, który ma przypisany określony identyfikator (ID) za pomocą atrybutu id.

Przykład

HTML:

```
<div id="main">
    To jest główny element
</div>
```

CSS:

```
#main {
    background-color: yellow;
    padding: 20px;
}
```

Styl zostanie zastosowany tylko do elementu o ID równym main.

Grupowanie selektorów (np. h1, p)

- Umożliwia zastosowanie tego samego stylu do wielu różnych elementów.

Przykład

```
h1, p {
    color: green;
}
```

Ustawia zielony kolor tekstu zarówno dla wszystkich elementów <h1>, jak i <p>.

Lektura uzupełniająca: W3Schools / CSS Selectors.

4.2. Selekcja oparta na relacji

Selektor potomków (spacja)

- Wybiera wszystkie elementy potomne danego elementu.

Przykład

```
div p {
    color: red;
}
```

Stylizuje wszystkie elementy <p>, które znajdują się wewnątrz elementu .

Selektor bezpośredniego potomka (>)

- Wybiera elementy, które są bezpośrednimi dziećmi danego elementu.

Przykład

```
div > p {
    color: blue;
}
```

Stylizuje tylko te elementy <p>, które są bezpośrednio wewnątrz .

Selektor sąsiedniego rodzeństwa (+)

- Wybiera element, który znajduje się bezpośrednio po innym elemencie na tym samym poziomie hierarchii.

Przykład

```
h1 + p {
    font-size: 18px;
}
```

Zmienia rozmiar czcionki pierwszego elementu <p> po każdym <h1>.

Selektor ogólnego rodzeństwa (~)

- Wybiera wszystkie elementy będące rodzeństwem po wskazanym elemencie.

Przykład

```
h1 ~ p {
    color: gray;
}
```

Zmienia kolor wszystkich elementów <p>, które są rodzeństwem <h1>.

Lektura uzupełniająca: W3Schools / CSS Combinators.

4.3. Pseudoklasy

Pseudoklasy dynamiczne

- Stylizują elementy w zależności od interakcji użytkownika:
  - :hover – styl dla elementu, gdy użytkownik najeżdża na niego myszką,
  - :focus – styl dla elementu, który ma fokus (np. aktywne pole formularza),
  - :active – styl dla elementu w momencie klikania.

Przykład

```
button:hover {
    background-color: lightblue;
}
```

Zmienia kolor tła przycisku, gdy użytkownik najeżdża na niego kursorem.

Pseudoklasy strukturalne

- Pozwalają stylizować elementy na podstawie ich pozycji w dokumencie:
  - :first-child – styl dla pierwszego dziecka w danym rodzicu,
  - :last-child – styl dla ostatniego dziecka w danym rodzicu,
  - :nth-child(n) – styl dla dziecka na określonej pozycji,
  - :nth-last-child(n) - jak wyżej, ale pozycja jest wyliczana od ostatniego elementu, a nie od pierwszego.

Przykład

```
li:nth-child(odd) {
    background-color: #f0f0f0;
}
```

Stylizuje co drugi element <li> (zaczynając od pierwszego).

Warto tutaj nadmienić, że możliwości pseudoklasy :nth-child(n) pozwalają na wybieranie stylowanych elementów na różne sposoby.

Przykład

- :nth-child(2n+3) - wybiera co drugi element, zaczynając od trzeciego,
- :nth-child(-2n+3) - wybiera co drugi element ale wstecz (!), zaczynając od trzeciego,
- :nth-child(4) - wybiera wyłącznie czwarty element.

Lektura uzupełniająca: W3Schools / CSS Pseudo-classes.

4.4. Pseudoelementy

Pseudoelementy ::before i ::after

- Dodają treść przed lub po zawartości elementu.

Przykład

```
p::before {
    content: ">> ";
    color: red;
}
```

Dodaje tekst >> przed każdym akapitem.

Pseudoelement ::selection

- Stylizuje część zawartości elementu, która jest zaznaczona przez użytkownika.

Przykład

```
p::selection {
  color: blue;
  background: gray;
}
```

Ustawia niebieski kolor czcionki oraz szare tło dla zaznaczonego tekstu.

Pseudoelement ::first-letter

- Stylizuje pierwszą literę zawartości elementu.

Przykład

```
p.intro::first-letter {
  color: #ff0000;
  font-size: 200%;
}
```

Ustawia pierwszą literę akapitu należącego do klasy "intro" na kolor czerwony oraz powiększa ją.

Lektura uzupełniająca: W3Schools / CSS Pseudo-elements

4.5. Selektory atrybutów

CSS3 umożliwia wybieranie elementów do stylizowania na podstawie ich atrybutów. Poniżej zamieszczono kilka przykładowych selektorów.

Selektor [attr]

- Wybiera elementy posiadające dany atrybut.

Przykład

```
input[type] {
    border: 1px solid blue;
}
```

Selektor [attr="value"]

- Wybiera elementy o określonej wartości atrybutu.

Przykład

```
input[type="text"] {
    background-color: yellow;
}
```

Selektor [attr^="value"]

- Wybiera elementy, których wartość atrybutu zaczyna się od określonego ciągu.

Przykład

```
a[href^="https://"] {
    color: green;
}
```

Selektor [attr$="value"]

- Wybiera elementy, których wartość atrybutu kończy się określonym ciągiem.

Przykład

```
a[href$=".pdf"] {
    text-decoration: underline;
}
```

Lektura uzupełniająca: W3Schools / CSS Attribute Selectors.

4.6. Filtry

Filtry w CSS3 są narzędziem pozwalającym na stosowanie efektów graficznych do elementów HTML (np. obrazów, tekstu czy innych elementów). Filtry mogą zmieniać wygląd elementów poprzez modyfikację ich kolorów, ostrości, kontrastu, jasności i innych właściwości wizualnych.

Filtry są dostępne dzięki właściwości CSS filter i mogą być stosowane zarówno do elementów rastrowych (np. obrazów), jak i wektorowych (np. elementów SVG), a także innych elementów HTML.

Ogólna składnia wygląda następująco:

```
selektor {
    filter: efekt1(wartość) efekt2(wartość) ...;
}
```

Dostępne filtry

- `filter: blur(5px);`
  Efekt rozmycia (ang. blur) powoduje, że element staje się niewyraźny. Wartość określa promień rozmycia w pikselach.
- `filter: contrast(1.5);`
  Zmienia kontrast elementu. Wartość domyślna to 1. Wartości powyżej 1 zwiększają kontrast. Wartości poniżej 1 (w przedziale do zera) zmniejszają kontrast.
- `filter: brightness(0.5);`
  Zmienia jasność elementu. Wartość domyślna to 1. Wartości powyżej 1 zwiększają jasność. Wartości poniżej 1 (w przedziale do zera) zmniejszają jasność.
- `filter: grayscale(1);`
  Przekształca element w odcienie szarości. Wartość to liczba z zakresu od 0 (brak efektu) do 1 (pełna skala szarości), lub analogicznie od 0% do 100%.
- `filter: invert(1);`
  Odwraca kolory elementu, dając efekt negatywu. Można ustawić głębokość efektu za pomocą liczby z zakresu 0 ... 1 (lub 0% ... 100%).
- `filter: opacity(0.5);`
  Zmienia przezroczystość elementu. Wartość to liczba z zakresu od 0 (całkowicie przezroczysty) do 1 (brak przezroczystości).
- `filter: saturate(2);`
  Zmienia nasycenie kolorów elementu. Wartości z przedziału 0 ... 1 pozwalają na zmniejszenie nasycenia, a wartości powyżej 1 zwiększają nasycenie.
- `filter: sepia(0.7);`
  Nadaje elementowi efekt sepii, który przypomina starą fotografię. Wartość to liczba z zakresu od 0 (brak efektu) do 1 (pełny efekt sepii).
- `filter: hue-rotate(90deg);`
  Zmienia barwy elementu poprzez obrót w kole barw o określony kąt (w stopniach). Wartość 0deg oznacza brak zmiany, a wartość maksymalna to 360deg.
- `filter: drop-shadow(offset-x offset-y blur-radius color);`
  przykład: `filter: drop-shadow(10px 10px 5px rgba(0, 0, 0, 0.5));`
  Dodaje cień do elementu.

Lektura uzupełniająca: https://www.w3schools.com/css/css3_image_filters.asp

5. Multimedia w aplikacji internetowej

Pod pojęciem *multimedia* kryje się sposób prezentacji treści, który łączy różne formy przekazu, takie jak tekst, dźwięk, obraz, animacje, wideo oraz interakcje. Dzięki temu użytkownik może doświadczać treści w sposób bardziej atrakcyjny i zrozumiały.

Najpopularniejsze elementy multimediów:

1. **Tekst** - podstawowy nośnik informacji, zwykle używany w połączeniu z innymi mediami.
2. **Grafika** - obrazy statyczne, takie jak zdjęcia, rysunki czy wykresy.
3. **Dźwięk** - nagrania dźwiękowe, muzyka lub narracja.
4. **Wideo** - materiały w formie krótszych lub dłuższych filmów, zazwyczaj z towarzyszącą ścieżką dźwiękową.
5. **Animacja** - ruchome obrazy, które pomagają wizualizować procesy lub dodawać dynamiki treści.
6. **Interaktywność** - elementy umożliwiające użytkownikowi interakcję z treścią.

O ile przekazywanie i formatowanie **tekstu** za pomocą języka HTML jest dość naturalne i zwykle nie sprawia większych problemów, o tyle pozostałe elementy multimedialne wymagają zdecydowanie większej uwagi.

5.1. Osadzanie filmów i nagrań audio

W języku HTML5 można łatwo umieścić odtwarzacz plików wideo lub audio na stronie za pomocą elementu <video>. Znacznik ten umożliwia odtwarzanie plików multimedialnych bez potrzeby stosowania dodatkowych wtyczek, takich jak Flash Player (co było powszechną praktyką przed laty). Możesz również dodać kontrolki, napisy (subtitles), i obsługiwać różne formaty plików.

Przykład

Odtwarzacz plików wideo możesz osadzić na stronie internetowej w następujący sposób:

```
<video controls autoplay loop muted poster="miniatura.jpg" width="640" height="360">
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    Twoja przeglądarka nie obsługuje wideo HTML5.
</video>
```

Przykład

Film wideo możesz wzbogacić o ścieżkę z napisami w różnych wersjach językowych w następujący sposób:

```
<video controls width="640" height="360">
    <source src="video.mp4" type="video/mp4">
    <track src="napisy.vtt" kind="subtitles" srclang="pl" label="Polski">
    <track src="napisy_en.vtt" kind="subtitles" srclang="en" label="English">
    Twoja przeglądarka nie obsługuje wideo HTML5.
</video>
```

Przykład

Odtwarzacz nagrań dźwiękowych osadzisz na stronie w następujący sposób:

```
<audio controls>
    <source src="muzyka.mp3" type="audio/mpeg">
    Twoja przeglądarka nie obsługuje dźwięku HTML5.
</audio>
```

5.2. Formaty plików graficznych

Wybór odpowiedniego formatu pliku graficznego do wykorzystania w nowoczesnej aplikacji internetowej zależy od rodzaju wyświetlanych obrazów (np. fotografie, schematy, zrzuty ekranu) oraz wymagań dotyczących jakości, wydajności i kompatybilności z przeglądarkami. W niniejszym artykule zebrano najpopularniejsze formaty graficzne oraz rekomendacje dotyczące ich zastosowań.

Format pliku graficznego w zależności od treści

Rodzaj obrazu	Rekomendowany format	Dlaczego?
Fotografie	WebP, JPEG, AVIF	Optymalna kompresja przy zachowaniu wysokiej jakości.
Schematy i logotypy	SVG	Skalowalność i mały rozmiar dla prostych grafik.
Zrzuty ekranu	PNG, WebP	Bezstratna kompresja i precyzja w przedstawieniu szczegółów.
Animacje	GIF, WebP, APNG, Lottie	Są to formaty wspierające proste animacje.

6. Język JavaScript

JavaScript to jeden z najpopularniejszych języków programowania wykorzystywanych w aplikacjach internetowych. Jest to **język skryptowy**, który może być uruchamiany tak po stronie front-endu (czyli przez przeglądarkę internetową za pomocą wbudowanego w nią interpretera), jak i po stronie back-endu (czyli na serwerze, w środowisku takim jak np. Node.js). JavaScript pozwala na tworzenie dynamicznych, interaktywnych i responsywnych elementów stron internetowych. W praktyce oznacza to, że JavaScript umożliwia użytkownikom wykonywanie różnych akcji na stronie internetowej bez konieczności jej przeładowania. Wśród przykładowych zastosowań można wskazać:

1. Animacje i efekty wizualne - tworzenie animowanych przejść, karuzel zdjęć, efektów przewijania,
2. Obsługa formularzy i interakcji z użytkownikiem - walidacja danych, dynamiczne dodawanie pól formularza,
3. Tworzenie dynamicznych treści - pobieranie danych z serwera bez przeładowania strony (AJAX, Fetch API),
4. Gry w przeglądarce - silniki do renderowania gier, w tym 3D, np. przy użyciu WebGL.

Język JavaScript charakteryzuje się szerokim ekosystemem narzędzi rozszerzających możliwości języka tudzież znakomicie ułatwiających pracę programisty poprzez zaimplementowanie rozwiązań najczęściej napotykanych problemów. Wśród najpopularniejszych narzędzi obecnie (rok 2024) wskazać należy:

- Frameworki: Angular, Vue.js,
- Biblioteki: React, jQuery, D3.js, Three.js.

Optymalny wybór konkretnego rozwiązania zależy zarówno od specyfiki danej aplikacji jak i od preferencji projektanta, gdyż funkcjonalności frameworków częściowo pokrywają się.

W zasadzie nic nie stoi na przeszkodzie, aby skrajnie proste aplikacje oprogramować z wykorzystaniem tzw. "surowego" JavaScriptu, tj. bez wykorzystania wyżej wymienionych rozszerzeń. Takie podejście warto rozważyć, gdy możliwe jest uzyskanie analogicznych funkcjonalności przy zbliżonym nakładzie pracy programisty, przy jednoczesnej chęci minimalizacji czasu ładowania aplikacji lub rozmiaru przesyłanych poprzez sieć plików. Jednakże współcześnie takie sytuacje zdarzają się raczej dość rzadko - głównie z uwagi na dość duże wymagania w zakresie odpowiednio zaawansowanych zachowań interfejsu aplikacji w różnych sytuacjach oraz powszechną dostępność szerokopasmowego dostępu do sieci Internet.

Istnieje jednak poważny argument stojący za tym, aby w swojej karierze popełnić jakąś aplikację w tzw. "surowym" JS. Mianowicie pozwala to zapoznać się z możliwościami i konstrukcjami oferowanymi przez ten język w oderwaniu od "bonusów", które niosą ze sobą frameworki i biblioteki. Dzięki temu w przyszłości łatwiej będzie opanować nowy framework/bibliotekę z uwagi na jednoznaczne rozróżnianie elementów składni kodu wnoszonych przez rozszerzenia względem naturalnej składni JavaScriptu.

6.1. Biblioteka jQuery

jQuery to popularna biblioteka JavaScript, która została stworzona w 2006 roku przez Johna Resiga. Jego celem jest uproszczenie pracy z językiem JavaScript, w szczególności manipulacji DOM, obsługi zdarzeń, animacji i wykonywania żądań AJAX. jQuery był jednym z najczęściej używanych narzędzi w tworzeniu dynamicznych aplikacji internetowych, w szczególności przed pojawieniem się bardziej zaawansowanych frameworków, takich jak React, Vue.js czy Angular.

Wraz z rozwojem nowoczesnego JavaScript (ES6+), jQuery jest używane coraz rzadziej. Wynika to ze stopniowego zanikania problemów, które jQuery z sukcesem rozwiązywało w momencie tworzenia:

- niejednolite wsparcie dla poszczególnych elementów składni JavaScript, HTML i CSS w różnych przeglądarkach - jQuery zapewniało programiście uporządkowany interfejs, a biblioteka zajmowała się obsługą różnic pomiędzy przeglądarkami,
- nowoczesny język JavaScript posiada funkcjonalności niegdyś dostępne dopiero z użyciem jQuery.

Lektura uzupełniająca:

- Strona projektu jQuery,
- Strona projektu jQuery UI (jest to biblioteka komponentów ułatwiających budowę interfejsu użytkownika).

6.2. React

React to popularna biblioteka JavaScript stworzona przez Facebook (obecnie Meta) w 2013 roku. Jest przeznaczona do budowania interfejsów użytkownika, w szczególności w aplikacjach jednostronicowych (ang. *Single Page Applications*, SPA). React pozwala budować dynamiczne i interaktywne interfejsy użytkownika na bazie komponenty, które mogą być zaprojektowane samodzielnie lub pochodzić z szerokiej bazy rozszerzeń.

React używa deklaratywnego stylu programowania, co oznacza, że deweloperzy opisują, jak interfejs użytkownika ma wyglądać, a React sam zarządza aktualizacjami w przeglądarce, minimalizując bezpośrednią manipulację DOM.

Jak zacząć?

Aby rozpocząć swoją przygodę z Reactem proponuję przerobić sobie następujące tutoriale:

1. Using React in Visual Studio Code
   - Edytor Visual Studio Code jest współcześnie zalecany jako podstawowe narzędzie deweloperskie.
   - Niniejszy tutorial pokazuje, w jaki sposób zacząć korzystać ze środowiska VSC na przykładzie prostego projektu startowego create-react-app.
   - Pamiętaj, że na samym początku potrzebujesz mieć zainstalowane środowisko NodeJS oraz menedżer pakietów npm w najnowszych stabilnych wersjach.
   - Czytelnik tutoriala prowadzony jest kolejno przez:
     - utworzenie nowego projektu,
     - podstawowe operacje w edytorze VSC,
     - skonfigurowanie debuggera oraz lintera.
   - Tyle na dobry początek. Następnie wypadałoby zacząć zgłębiać tajniki samego Reacta...
2. Tutorial: Tic-Tac-Toe
   - Ten tutorial prowadzi Czytelnika przez proces stopniowego budowania prostej gry typu "kółko i krzyżyk" działającej wyłącznie po stronie klienta.
   - W kolejnych krokach wprowadzany jest deklaratywny sposób opisu komponentów, elementy składni JSX, zarządzanie stanem itd.
   - Choć opisane w tutorialu kroki można ćwiczyć w dostępnym na stronie działającym online sandboksie, gorąco zachęcam Ciebie do pobrania sobie początkowych plików projektu i stopniowe rozwijanie go w lokalnie skonfigurowanym środowisku.
   - Pamiętaj, aby nie przeklejać (Ctrl+C / Ctrl+V) fragmentów kodu z tutoriala, ale samodzielnie wpisywać go w edytorze - to ułatwia zapamiętanie.
3. Thinking in React
   - Kolejny tutorial prowadzi do zbudowania złożonego interfejsu użytkownika aplikacji webowej i jest trampoliną do realizacji własnych projektów w kolejnych krokach.

Popularne rozszerzenia do Reacta

- React Router
  Umożliwia dodanie routingu do aplikacji React.

- Redux
  Biblioteka do zarządzania stanem aplikacji.

- React Query
  Obsługa zapytań do API i zarządzanie danymi asynchronicznymi.

- Formik
  Ułatwia obsługę i walidację formularzy.

- React Testing Library
  Narzędzie do testowania komponentów React.

7. Responsive Web Design

Responsywny projekt stron internetowych (ang. *Responsive Web Design*, *RWD*), to podejście do projektowania stron i aplikacji internetowych, które zakłada automatyczne dostosowywanie ich wyglądu i układu treści do parametrów ekranów różnych urządzeń, na których aplikacja będzie prezentowana. Głównym celem RWD jest zapewnienie użytkownikom optymalnego doświadczenia – wygodnego przeglądania i łatwej nawigacji, niezależnie od tego, czy korzystają z komputera, tabletu, czy smartfona.

Zalety Responsive Web Design:

1. Jedna wersja strony dla wszystkich urządzeń: strona przygotowana zgodnie z RWD dostosowuje się automatycznie do różnych ekranów, co eliminuje konieczność tworzenia osobnych wersji np. mobilnej i desktopowej.
2. Optymalizacja kosztów i łatwiejsze utrzymanie: zarządzanie jedną stroną responsywną jest łatwiejsze i bardziej opłacalne niż obsługa wielu wersji.
3. Lepsze doświadczenie użytkownika (UX): responsywne strony są czytelne i łatwe w nawigacji na każdym urządzeniu. Przy poprawnym projekcie nie ma potrzeby przewijania w poziomie czy powiększania zawartości na małych ekranach.
4. Lepsza widoczność w wyszukiwarkach (SEO): Google preferuje responsywne strony, co wpływa na ich wyższą pozycję w wynikach wyszukiwania.
5. Przyszłościowe podejście: strony responsywne są gotowe na różne urządzenia, nawet te, które jeszcze nie istnieją (np. nowe typy ekranów).

Wady Responsive Web Design:

1. Większa złożoność projektu: stworzenie strony responsywnej wymaga dodatkowych nakładów pracy w zakresie planowania i testowania.
2. Możliwe problemy z wydajnością: przy błędnej implementacji responsywna strona może ładować wszystkie zasoby (np. obrazy w dużych rozdzielczościach) na każde urządzenie, co może w zauważalny sposób obciążać urządzenia o ograniczonych zasobach.
3. Nie wszystkie strony nadają się do RWD: w niektórych przypadkach (np. bardzo skomplikowanych aplikacji) może być korzystniejsze stworzenie osobnej wersji mobilnej i desktopowej.

Obecnie (rok 2024), z uwagi na ogromną popularność urządzeń przenośnych do przeglądania sieci Internet (smartfony, tablety) przy wciąż dużym udziale komputerów stacjonarnych, stosowanie podejścia responsywnego jest w zasadzie niezbędne aby zapewnić odpowiednio wysoką jakość obsługi szerokiego grona użytkowników aplikacji internetowych.

7.1. Stopniowe ulepszanie strony

Koncepcja stopniowego ulepszania strony (ang. *Progressive Enhancement*) dotyczy projektowania i tworzenia stron internetowych w sposób, który zapewnia ich podstawową funkcjonalność i dostępność dla wszystkich użytkowników, niezależnie od wykorzystywanego urządzenia czy przeglądarki, z jednoczesnym rozbudowywaniem interfejsu użytkownika i udostępnianiem bardziej zaawansowanych funkcji dla nowoczesnych przeglądarek i urządzeń.

Stopniowe ulepszanie często jest praktykowane w połączeniu z Responsive Web Design (RWD). Okazuje się, że zdecydowanie łatwiej jest zapewnić responsywność interfejsu użytkownika przy takim sposobie myślenia i projektowania UI, niż przy podejściu odwrotnym, czyli takim, w którym najpierw projektuje się interfejs zawierający wszystkie funkcjonalności a następnie próbuje się go upraszczać w celu dostosowania do możliwości prostszych urządzeń.

Praktyczny przykład budowy fragmentu prostego interfejsu aplikacji zgodnie z koncepcją stopniowego ulepszania

1. Stwórz solidny fundament w języku HTML

Przykład

Treść strony i jej struktura powinny być dostępne i funkcjonalne w podstawowej przeglądarce obsługującej tylko HTML.

```
<nav>
    <ul>
        <li><a href="#home">Strona główna</a></li>
        <li><a href="#about">O nas</a></li>
        <li><a href="#contact">Kontakt</a></li>
    </ul>
</nav>
```

Nawet bez CSS i JavaScript menu nawigacyjne jest dostępne i funkcjonalne.

2. Dodaj stylizację za pomocą CSS

Przykład

Stosuj CSS, aby poprawić wygląd strony, jednocześnie upewniając się, że treść pozostaje czytelna nawet w przypadku przeglądarek nieobsługujących CSS lub przy wyłączonym stylowaniu.

```
nav ul {
    list-style: none;
    display: flex;
    gap: 10px;
}

nav ul li a {
    text-decoration: none;
    color: #007bff;
}
```

3. Zaimplementuj responsywność za pomocą zapytań medialnych CSS

Przykład

Zapytania medialne CSS pozwalają dostosować wygląd strony do różnych rozdzielczości ekranu.

```
@media (max-width: 768px) {
    nav ul {
        flex-direction: column;
    }
}
```

Nawigacja zmieni układ z poziomego na pionowy, jeśli szerokość ekranu wynosi 768px lub mniej.

4. Dodaj interaktywność za pomocą JavaScriptu

Przykład

JavaScript może być użyty do wprowadzenia dodatkowej interaktywności, np. rozwijanego menu czy animacji.
Pamiętaj, aby JavaScript był opcjonalny – strona powinna działać nawet bez niego.

```
document.querySelector('.menu-toggle').addEventListener('click', function() {
    document.querySelector('nav ul').classList.toggle('show');
});
```

5. Użyj funkcji wykrywania wsparcia technologii (Feature Detection)

Przykład

Przed dodaniem zaawansowanych funkcji sprawdź, czy przeglądarka je obsługuje. Możesz użyć narzędzi takich jak **Modernizr** lub pisać własne sprawdzenia.

```
if ('IntersectionObserver' in window) {
    // Dodaj zaawansowaną funkcjonalność np. lazy loading obrazków
    console.log('IntersectionObserver jest obsługiwany');
} else {
    console.log('Brak wsparcia dla IntersectionObserver');
}
```

Czym jest **Intersection Observer API przeczytasz tutaj**.

8. Dostępność cyfrowa

Dostępność cyfrowa jest jedną z cech, jakimi powinny charakteryzować się rozwiązania cyfrowe, w tym aplikacje internetowe, aby umożliwić korzystanie z nich jak największej grupie użytkowników, w tym osobom z różnymi niepełnosprawnościami, np. wzroku, słuchu, ruchu, ale też z niepełnosprawnością intelektualną czy zaburzeniami poznawczymi. Część osób z niepełnosprawnościami korzysta z tzw. technologii asystujących (np. osoby niewidome wykorzystują oprogramowanie czytające na głos treści prezentowane w oknach przeglądarek internetowych), dzięki którym mają szansę w miarę swobodnie korzystać z nowoczesnych cyfrowych rozwiązań. Aby jednak te rozwiązania działały poprawnie, oferowane treści muszą zostać przygotowane w odpowiedni sposób, uwzględniający nie tylko tradycyjne aspekty wizualne, ale również m.in. odpowiednią strukturę logiczną interfejsu aplikacji.

Elementy niezbędne dla osób z niepełnosprawnościami w wielu przypadkach okażą się przydatne także dla pozostałych użytkowników. Odpowiedni kontrast tekstu względem tła wspiera odczytanie treści przez osoby słabowidzące, ale również starsze czy korzystające z urządzeń mobilnych. Napisy dodane do filmu wykorzystają zarówno osoby słabosłyszące, jak i słabiej znające język czy oglądające film w głośnym otoczeniu.

W Polsce pierwszym aktem prawnym w całości dotyczącym dostępności cyfrowej jest Ustawa z dnia 4 kwietnia 2019 r. o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych. Wdraża ona wyżej wymienioną dyrektywę PE 2016/2102, nakłada na podmioty publiczne obowiązek zapewnienia dostępności cyfrowej i obowiązek zamieszczenia deklaracji dostępności, a także określa zasady monitorowania dostępności cyfrowej i zasady postępowania w przypadku jej braku. Wymagania, które musi spełniać strona internetowa i aplikacja mobilna dostępna cyfrowo, zostały zebrane zebrane w tabeli, która jest załącznikiem do tej ustawy. Tabela ta (z niewielkimi ograniczeniami) jest równoważna wytycznym WCAG 2.1 na poziomie AA. Do tego każda strona internetowa i każda aplikacja mobilna objęta wymaganiami ustawy musi być posiadać deklarację dostępności. Ustawa dotyczy także stron internetowych działających wewnętrznie (np. usługi typu intranet).

Także Ustawa z dnia 19 lipca 2019 r. o zapewnianiu dostępności osobom ze szczególnymi potrzebami w pewnych kwestiach również odnosi się do dostępności cyfrowej. Jej celem jest poprawa warunków życia i funkcjonowania obywateli ze szczególnymi potrzebami, m.in. ze względu na niepełnosprawność lub obniżony poziom sprawności z powodu wieku czy choroby.

Obszerne omówienie zagadnień dostępności cyfrowej znajduje się na stronach rządowych. Poniżej zamieszczam kilka miejsc, do których warto zajrzeć na samym początku:

- Co to jest dostępność cyfrowa
- Jakie akty prawne dotyczą dostępności cyfrowej
- Omówienie wymogów dostępności cyfrowej dla podmiotów publicznych
- Jakie są sposoby wyszukiwania błędów i badania dostępności cyfrowej
- Jak automatycznie testować dostępność cyfrową stron internetowych
- Jakie elementy tworzą dostępność cyfrową na stronach internetowych

W kolejnych podrozdziałach tego podręcznika zapoznasz się:

- ze strukturą, zasadami i wymaganiami stawianymi przez rekomendacje WCAG,
- z rekomendacją WAI-ARIA, która rozszerza semantykę języka HTML o dodatkowe atrybuty ułatwiające implementację rozwiązań wspierających dostępność,
- z narzędziami do walidacji stron i aplikacji pod kątem ich dostępności cyfrowej,
- z przykładami wdrożeń rozwiązań z zakresu dostępności cyfrowej.

8.1. Zalecenia WCAG

WCAG (ang. *Web Content Accessibility Guidelines*) to zbiór międzynarodowych zaleceń opracowanych przez organizację W3C (World Wide Web Consortium), które mają na celu zwiększenie dostępności stron i aplikacji internetowych dla jak najszerszego grona użytkowników, w tym osób z niepełnosprawnościami. WCAG wskazuje, jak projektować i budować treści cyfrowe, aby były bardziej dostępne i przyjazne dla osób z różnymi ograniczeniami, takimi jak:

- niepełnosprawność wzrokowa (np. osoby niewidome lub słabowidzące),
- niepełnosprawność słuchowa,
- trudności w poruszaniu się (np. osoby korzystające z urządzeń wspomagających),
- trudności poznawcze (np. dysleksja, zaburzenia koncentracji),
- osoby starsze, które mogą mieć ograniczone zdolności sensoryczne lub motoryczne,
- osoby korzystające z nietypowych urządzeń (np. telefonów, czytników ekranowych).

Zalecenia WCAG mają na celu:

- zapewnienie dostępności dla wszystkich użytkowników, niezależnie od ich ograniczeń,
- zwiększenie użyteczności stron internetowych i aplikacji,
- zapewnienie zgodności ze standardami prawa dotyczącymi dostępności cyfrowej, np. w krajach Unii Europejskiej obowiązuje dyrektywa 2016/2102 o dostępności cyfrowej (*Dyrektywa Parlamentu Europejskiego i Rady z 26 października 2016 r. o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów sektora publicznego*).

Regulacje prawne mające zastosowanie w Polsce zostały zebrane w następującym artykule: Jakie akty prawne dotyczą dostępności cyfrowej.

Zapoznaj się: Oficjalne tłumaczenie wytycznych WCAG 2.1. dla dostępności treści internetowych

Wersje rekomendacji WCAG (stan na grudzień 2024):

- WCAG 2.0 opublikowano 11. grudnia 2008 roku,
- WCAG 2.1 opublikowano 5. czerwca 2018 roku, z aktualizacjami 21. września 2023 i 12. grudnia 2024,
- WCAG 2.2. opublikowano 5. października 2023 roku, z aktualizacją 12. grudnia 2024.

(źródło)

8.2. Kluczowe zasady WCAG - "POUR"

Zalecenia WCAG opierają się na czterech głównych zasadach, które można łatwiej zapamiętać korzystając z akronimu **POUR**:

1. **Postrzegalność (ang. *Perceivable*):**
   Zasada postrzegalności oznacza, że wszystkie informacje i komponenty interfejsu muszą być przedstawione w sposób, który użytkownik może odebrać za pomocą swoich zmysłów, takich jak wzrok, słuch lub dotyk. Treści muszą być dostępne dla różnych grup użytkowników, niezależnie od ich ograniczeń sensorycznych.
   Przykłady:
   - zapewnienie tekstów alternatywnych (atrybut *alt* dla elementu <img>) dla wszystkich treści multimedialnych (obrazy, wideo, itp.), aby osoby niewidome mogły je odczytać za pomocą czytników ekranowych,
     ```
     <img src="przyklad.jpg" alt="Zdjęcie Mostu Północnego o zachodzie słońca">
     ```
   - udostępnienie napisów (transkrypcji) do treści audio i wideo dla osób niesłyszących i niedosłyszących,
   - używanie odpowiednio wysokiego kontrastu między tekstem a tłem (min. 4.5:1 dla zwykłego tekstu i 3:1 dla dużych czcionek), aby treści były czytelne dla osób słabowidzących,
   - treści powinny być skalowalne, np. umożliwiać użytkownikom powiększanie tekstu (do 200%) bez utraty funkcjonalności lub czytelności.

2. **Funkcjonalność (ang. *Operable*):**
   Strona lub aplikacja powinna być łatwa w nawigacji i obsłudze, niezależnie od sposobu interakcji użytkownika (mysz, klawiatura, urządzenia wspomagające).
   Przykłady:
   - strony muszą być w pełni dostępne dla użytkowników korzystających wyłącznie z klawiatury (np. osoby z ograniczeniami ruchowymi, które nie używają myszy),
   - strona powinna mieć logiczną i intuicyjną strukturę nawigacyjną, np. menu, spis treści, widoczne linki "przejdź do treści" (skip to content), które pozwalają szybko dotrzeć do głównej części strony,
   - zapewnienie wystarczającego czasu na przeczytanie i zrozumienie treści, a także na interakcję (np. brak automatycznego zamykania ważnych informacji, możliwość przedłużenia czasu wypełniania formularza itp.),
   - unikanie elementów wywołujących epilepsję, takich jak migoczące światła lub animacje.

3. **Zrozumiałość (ang. *Understandable*):**
   Zasada jest taka, jak się wydaje - użytkownicy końcowi muszą być w stanie zrozumieć zawartość strony internetowej niezależnie od ich umiejętności poznawczych, językowych czy technicznych..
   Przykłady:
   - treści powinny być napisane prostym i zrozumiałym językiem, dostosowanym do docelowych użytkowników; w razie potrzeby należy unikać żargonu lub wyjaśniać trudne pojęcia,
   - zapewnienie instrukcji dotyczących obsługi formularzy (np. w przypadku błędu walidacji formularza wskazać, którego pola dotyczy błąd i jaka jest jego przyczyna),
   - treść strony powinna być oznaczona odpowiednim atrybutem języka (np. lang="pl", lang="en"), co pozwala czytnikom ekranowym poprawnie odczytywać teksty.

4. **Solidność (ang. *Robust*):**
   Treści powinny być kompatybilne z różnymi technologiami, w tym z obecnymi i przyszłymi narzędziami wspomagającymi (np. czytnikami ekranowymi).
   Przykłady:
   - kod HTML powinien być zgodny ze standardami W3C, aby technologie wspomagające, takie jak czytniki ekranowe, mogły poprawnie interpretować strukturę strony; przykładowo należy korzystać z semantycznych znaczników HTML5, takich jak <header>, <main>, <footer>, zamiast generować strukturę strony wyłącznie za pomocą znaczników ,
   - elementy interaktywne (np. przyciski, linki, formularze) powinny być dostępne dla użytkowników korzystających z technologii wspomagających, takich jak czytniki ekranowe, lupy ekranowe czy przełączniki,
   - należy unikać implementacji niestandardowych zachowań elementów interfejsu, takich jak przyciski, linki czy pola formularzy - one powinny działać zgodnie z ich naturalnym przeznaczeniem.

8.3. Jak uzyskać zgodność?

Co oznacza zgodność?

Zgodność ze standardem oznacza, że spełniasz „wymagania" standardu. W WCAG 2 „wymagania" są kryteriami sukcesu. Aby zachować zgodność z WCAG 2, musisz spełnić kryteria sukcesu, to znaczy, że nie mogą istnieć treści, które nie spełniają któregoś z kryteriów sukcesu. Oznacza to, że jeśli nie ma treści, do której odnosi się dane kryterium sukcesu, jest spełnione z automatu.

Zdefiniowanych jest **pięć wymagań**, które omówione zostaną poniżej wraz z towarzyszącymi im niektórymi pojęciami na kolejnych stronach.

Wymaganie nr 1 (poziomy zgodności)

Pierwsze wymaganie dotyczy poziomów zgodności. Zasadniczo mówi, że wszystkie informacje na stronie są zgodne lub posiadają zgodną wersję alternatywną, która jest dostępna ze strony. Wymaganie wyjaśnia również, że żadna zgodność nie jest możliwa bez spełnienia przynajmniej wszystkich kryteriów sukcesu poziomu A.

Poziomy zgodności

WCAG2 definiuje trzy poziomy zgodności, a więc i trzy poziomy kryteriów sukcesu:

- Poziom A: strona internetowa spełnia wszystkie kryteria sukcesu poziomu A lub zapewniona jest **zgodna wersja alternatywna**,
- Poziom AA: strona internetowa spełnia wszystkie kryteria sukcesu poziomu A i poziomu AA lub zapewniona jest wersja alternatywna zgodna z poziomem AA,
- Poziom AAA: strona internetowa spełnia wszystkie kryteria sukcesu poziomu A, poziomu AA i poziomu AAA lub zapewniona jest wersja alternatywna zgodna z poziomem AAA.

Poziom AAA stawia najwyższe wymagania. Nie zaleca się, aby zgodność z poziomem AAA była wymagana jako ogólna polityka dla całych witryn, ponieważ nie jest możliwe spełnienie wszystkich kryteriów sukcesu poziomu AAA dla niektórych treści.

Więcej szczegółów: WCAG 2.2 Understanding Docs / Understanding Requirement 1

Wymaganie nr 2 (zgodność dla całych witryn)

Zgodność (i poziom zgodności) dotyczy tylko pełnych stron internetowych i nie może zostać osiągnięta, jeśli część strony internetowej jest z niej wyłączona.

- Do celów określenia zgodności, alternatywy dla części treści strony są uważane za część strony, gdy do wersji alternatywnych można uzyskać dostęp bezpośrednio ze strony, np. długi opis ilustracji lub alternatywna opis materiału wideo.
- Autorzy stron internetowych, które nie mogą być zgodne z wymaganiami ze względu na zawartość pozostającą poza kontrolą autora, mogą rozważyć złożenie "Oświadczenia o częściowej zgodności".
- Pełna witryna obejmuje każdą odmianę strony, która jest automatycznie prezentowana dla różnych rozmiarów ekranu (np. odmiany na responsywnej stronie internetowej). Każda z tych odmian musi być zgodna (lub musi mieć zgodną wersję alternatywną), aby cała strona mogła zostać uznana za zgodną.

Niniejsza reguła wymaga po prostu, aby cała witryna była zgodna. Tym samym nie można stosować oświadczeń o "zgodności części strony". Istnieją jednak pewne wyjątki od tej reguły.

Czasami informacje uzupełniające mogą być dostępne na innej stronie. Przykładem jest atrybut `longdesc` w języku HTML. Dzięki `longdesc` długi opis grafiki może znajdować się na osobnej stronie, do której użytkownik może przejść ze strony z grafiką. Wyjaśnia to, że taka zawartość jest uważana za część witryny internetowej, więc wymaganie nr 2 jest spełnione dla połączonego zestawu stron internetowych traktowanych jako pojedyncza witryna internetowa. Alternatywy mogą być również udostępniane na tej samej stronie, na przykład tworząc odpowiednik interfejsu użytkownika.

Ze względu na wymaganie zgodności nr 5, cała witryna może być uznana za zgodną, nawet jeśli jej części wykorzystują technologie nieobsługujące dostępności, o ile nie kolidują one z resztą strony, a wszystkie informacje i funkcje są dostępne w innym miejscu na stronie lub z jej poziomu.

Możliwe jest dołączenie treści niezgodnych - zobacz objaśnienie do wymagania nr 5.

Wymaganie nr 3 (cały proces)

Gdy strona internetowa jest jedną z serii stron internetowych przedstawiających jakąś procedurę (tj. sekwencję kroków, które należy wykonać w celu wykonania danej czynności), wszystkie strony internetowe prezentujące etapy procedury są zgodne na określonym poziomie lub lepszym. (Innymi słowy, zgodność nie zostanie osiągnięta na danym poziomie, jeśli którakolwiek ze stron prezentujących etap procedury nie jest zgodna na tym samym poziomie lub lepszym).

Przykład: Sklep internetowy ma serię stron, które służą do wyboru i zakupu produktów. Wszystkie strony w procedurze zakupowej od początku (przeglądanie oferty, wrzucanie produktów do koszyka, itp.) do końca (kasa, płatność) muszą być zgodne, aby każda strona, która jest częścią tej procedury, mogła być uznana za zgodną.

Przepis ten uniemożliwia uznanie za zgodną strony internetowej, która jest częścią większej procedury, jeśli cała procedura nie jest zgodna. Uniemożliwia to sklasyfikowanie sklepu internetowego jako zgodnego z wymaganiami dostępności, jeśli np. kasa lub jakaś inna funkcjonalność witryny będąca częścią procedury zakupowej nie jest zgodna.

Wymaganie nr 4 (technologie obsługujące dostępność)

Aby spełnić kryteria sukcesu, należy polegać wyłącznie na **obsługujących dostępność** sposobach korzystania z technologii. Wszystkie informacje lub funkcje, które są zapewnione w sposób nieobsługujący dostępności, muszą być również zapewnione w sposób obsługujący dostępność.

Wiele z kryteriów sukcesu dotyczy zapewnienia dostępności za pomocą technologii wspomagających lub specjalnych funkcji dostępności w popularnych programach wykorzystywanych przez użytkownika (na przykład opcja „pokaż napisy” w odtwarzaczu multimedialnym). Oznacza to, że kryteria sukcesu wymagają, aby treści internetowe zostały przygotowane w taki sposób, który umożliwiłby technologiom wspomagającym pomyślne przedstawienie użytkownikowi zawartych w nich informacji. Na przykład obrazek, który należy kliknąć, aby przejść do danej podstrony, nie będzie dostępny dla osoby niewidomej, chyba że tekst alternatywny dla obrazka zostanie dostarczony w sposób umożliwiający programom użytkownika, w tym technologiom wspomagającym, ich znalezienie i wyświetlenie albo np. przeczytanie za pomocą syntezatora mowy. Kluczem jest tutaj to, że tekst alternatywny musi być dołączony w sposób, który programy użytkownika, w tym technologie wspomagające, mogą zrozumieć i wykorzystać - w sposób, który jest „Obsługiwany przez dostępność”.

Innym przykładem może być niestandardowa kontrolka umieszczona na stronie internetowej. W takim przypadku standardowy program użytkownika zwykle nie byłby w stanie przedstawić użytkownikowi alternatywy. Jeśli jednak informacje o kontrolce, w tym jej nazwa, rola, wartość, sposób jej ustawienia itp. są dostarczane w sposób, w jaki technologie wspomagające mogą je zrozumieć i kontrolować, wówczas użytkownicy z technologiami wspomagającymi będą mogli korzystać z tych kontrolek.

Gdy wprowadzane są nowe technologie, muszą wydarzyć się dwie rzeczy, aby osoby korzystające z technologii wspomagających mogły uzyskać do nich dostęp. Po pierwsze, technologie muszą być zaprojektowane w taki sposób, aby programy użytkownika, w tym technologie wspomagające, mogły uzyskać dostęp do wszystkich informacji potrzebnych do zaprezentowania treści użytkownikowi. Po drugie, programy użytkownika i technologie wspomagające mogą wymagać przeprojektowania lub modyfikacji, aby móc faktycznie pracować z tymi nowymi technologiami.

„Obsługiwanie dostępności” oznacza, że obie te czynności zostały wykonane, a technologia będzie współpracować z agentami użytkownika i technologiami wspomagającymi.

Więcej na ten temat przeczytasz tutaj: WCAG 2.2 Understanding Docs / Understanding Accessibility Support

Wymaganie nr 5 (bez zakłóceń)

Jeśli zastosowane na stronie technologie są użyte w sposób, który nie obsługuje dostępności lub jeśli są używane w sposób niezgodny z kryteriami sukcesu, wówczas nie blokują one użytkownikom możliwości dostępu do pozostałej części strony. Ponadto strona internetowa jako całość nadal spełnia wymagania zgodności jeśli spełniony jest każdy z poniższych warunków:

- gdy jakakolwiek technologia, na której nie można polegać, jest włączona w programie użytkownika,
- gdy jakakolwiek technologia, na której nie można polegać, jest wyłączona w programie użytkownika, **oraz**
- gdy jakakolwiek technologia, na której nie można polegać, nie jest obsługiwana przez program użytkownika.

Ponadto poniższe kryteria sukcesu mają zastosowanie do wszystkich treści na stronie, w tym treści, które w innych przypadkach nie są podstawą spełnienia wymogów zgodności, ponieważ niezastosowanie poniższych kryteriów mogłoby uniemożliwiać korzystanie ze strony:

- 1.4.2 - Sterowanie dźwiękiem,
- 2.1.2 - Brak pułapki klawiaturowej,
- 2.3.1 - Trzy mignięcia lub poniżej progu, oraz
- 2.2.2 - Pauza, Zatrzymanie, Ukrycie.

Uwaga: Jeśli strona nie może być zgodna (na przykład strona testowa zgodności lub strona przykładowa), nie może być uwzględniona w zakresie zgodności ani w oświadczeniu o zgodności.

Wymaganie to zasadniczo mówi, że można stosować technologie, które nie obsługują dostępności, o ile wszystkie informacje są również zapewnione przy użyciu technologii obsługujących dostępność i o ile materiały nieobsługiwane pod kątem dostępności nie powodują zakłóceń.

Mogą być używane technologie, które nie obsługują dostępności, lub technologie, które obsługują dostępność, ale używane w sposób niezgodny,

o ile tylko wszystkie informacje są również dostępne przy użyciu technologii obsługiwanych pod kątem dostępności, w sposób zgodny i o ile materiały nieobsługiwane pod kątem dostępności nie przeszkadzają.

Więcej na ten temat przeczytasz tutaj: WCAG 2.2 Understanding Docs / Understanding Requirement 5

8.4. Zgodna wersja alternatywna

Wymaganie zgodności nr 1 zezwala na zadeklarowanie zgodności dla niezgodnych stron, o ile mają one „zgodną wersję alternatywną". Definiuje się ją jako wersję, która:

1. jest zgodna na danym poziomie, **oraz**
2. zapewnia wszystkie te same informacje i funkcje w tym samym ludzkim języku, **oraz**
3. jest tak samo aktualna jak treść niezgodna, **oraz**
4. dla której co najmniej jeden z poniższych warunków jest prawdziwy:
    - do wersji zgodnej można dotrzeć z wersji niezgodnej za pomocą mechanizmu obsługującego dostępność, **lub**
    - do wersji niezgodnej można dotrzeć tylko z wersji zgodnej, **lub**
    - do wersji niezgodnej można dotrzeć tylko ze strony zgodnej, która zapewnia również mechanizm umożliwiający dotarcie do wersji zgodnej.

W tej definicji „można dotrzeć tylko" oznacza, że zapewniony został pewien mechanizm, jak np. warunkowe przekierowanie, który uchroni użytkownika przed "dotarciem do" (załadowaniem) niezgodnej wersji, chyba że użytkownik właśnie przeszedł z wersji zgodnej.

Zgodna wersja alternatywna nie musi być identyczna, co do ilości stron z wersją podstawową (np. może się składać z kilku stron).

Jeśli dostępnych jest wiele wersji językowych, dla każdego oferowanego języka wymagane są zgodne wersje alternatywne.

Wersje alternatywne mogą być zapewniane w celu dostosowania do różnych środowisk technologicznych lub grup użytkowników. Każda wersja powinna być zgodna w jak najwyższym stopniu. Przynajmniej jedna wersja musi być w pełni zgodna, aby spełnić wymaganie zgodności nr 1.

Zgodna wersja alternatywna nie musi znajdować się w zakresie zgodności, ani nawet na tej samej stronie internetowej, o ile jest równie łatwo dostępna jak wersja niezgodna.

Wersji alternatywnych nie należy mylić z treściami uzupełniającymi, które wspierają oryginalną stronę i poprawiają jej zrozumienie.

Ustawienie w preferencjach użytkownika, które pozwala na dotarcie do zgodnej wersji alternatywnej, jest akceptowalnym mechanizmem dotarcia do innej wersji, o ile mechanizm ustawień obsługuje dostępność.

Więcej o tym, dlaczego WCAG zezwala na wersje alternatywne witryn, a także o technikach zapewniania zgodnej wersji alternatywnej, przeczytasz tutaj: WCAG 2.2 Understanding Docs / Understanding Conforming Alternate Versions

8.5. Poziom wsparcia technologii wspomagających

Pojawia się więc pytanie, ile lub które technologie wspomagające muszą obsługiwać technologię internetową, aby ta technologia internetowa została uznana za „obsługującą dostępność". Grupa Robocza WCAG i W3C nie określają, które lub ile technologii wspomagających musi obsługiwać technologię internetową, aby można je było sklasyfikować jako obsługujące dostępność. Jest to złożony problem, który różni się zarówno w zależności od środowiska, jak i języka. Istnieje potrzeba zewnętrznego i międzynarodowego dialogu na ten temat. Oto kilka uwag, które pomogą w zrozumieniu i zgłębieniu tego tematu:

1. Wsparcie dostępności technologii internetowych różni się w zależności od środowiska.
    - Technologie internetowe mogą być obsługiwane tylko przez konkretne programy użytkownika i technologie wspomagające wdrożone w firmie (mogą to być starsze programów użytkownika i technologii wspomagających lub ich najnowsze wersje).
    - Treści publikowane w publicznej sieci Internet mogą wymagać współpracy z szerszym zbiorem programów użytkownika i technologii wspomagających, w tym ze starszymi wersjami.
2. Obsługa dostępności przez technologie internetowe różni się w zależności od języka (i dialektu).
    - Istnieją różne poziomy wsparcia starszych technologii wspomagających w różnych językach, a nawet krajach. Niektóre środowiska lub kraje mogą zapewniać bezpłatne technologie wspomagające.
3. Nowe technologie nie będą obsługiwane w starszych technologiach wspomagających.
    - Oczywiście nowa technologia nie może być obsługiwana przez wszystkie dotychczasowe technologie wspomagające, w związku z tym wymaganie, aby dana technologia była obsługiwana przez wszystkie technologie wspomagające, nie jest możliwe do spełnienia.
4. Wsparcie dla jednej starszej technologii wspomagającej zwykle nie jest wystarczające.
    - Obsługa tylko jednej technologii wspomagającej (dla danej niepełnosprawności) zwykle nie byłaby wystarczająca, zwłaszcza jeśli większość użytkowników, którzy potrzebują jej w celu uzyskania dostępu do treści, nie posiada takiej technologii wspomagającej i nie może sobie na nią pozwolić. Wyjątkiem są tutaj np. informacje rozpowszechniane tylko wśród pracowników firmy, gdzie wszyscy mają jedną technologię wspomagającą (tego typu).
5. Obecnie technologie wspomagające, które są dostępne dla ogółu społeczeństwa, są często niskiej jakości.
    - Należy unikać tworzenia treści, które nie mogą być używane przez ogół społeczeństwa z niepełnosprawnościami. W wielu przypadkach koszt technologii wspomagających jest zbyt wysoki dla użytkowników, którzy ich potrzebują. Ponadto możliwości bezpłatnych lub niedrogich technologii wspomagających są dziś często tak niewielkie, że treści internetowych nie można realistycznie ograniczyć do tego najniższego (lub nawet średniego) wspólnego mianownika. Stwarza to bardzo trudny dylemat, który należy rozwiązać.

Grupa Robocza ograniczyła się zatem do zdefiniowania, co stanowi obsługę (wsparcie) i odłożyła ocenę tego, ile lub które technologie wspomagające muszą obsługiwać daną technologię, pozostawia społeczności i podmiotom bliższym każdej sytuacji, które ustalają wymagania dla organizacji, zakupu, społeczności itp.

Więcej na ten temat przeczytasz tutaj: WCAG 2.2 Understanding Docs / Understanding Accessibility Support

8.6. Kryteria sukcesu

Zestawienie kryteriów sukcesu wraz z ich wyjaśnieniami znajdziesz na stronie How to Meet WCAG (Quick Reference). W szczególności można wybrać wersję zaleceń WCAG:

- WCAG 2.0,
  - wersja polskojęzyczna,
- WCAG 2.1,
  - wersja polskojęzyczna,
- WCAG 2.2,
  - wersja polskojęzyczna,
- kryteria sukcesu dodane w WCAG 2.1,
  - wersja polskojęzyczna,
- kryteria sukcesu dodane w WCAG 2.2,
  - wersja polskojęzyczna.

Aby przybliżyć sobie zagadnienia pokrywane przez kryteria sukcesu, warto zapoznać się z tzw. listą kontrolną WCAG 2.1 dla poziomu A oraz AA opublikowaną na stronie Smultron Web Development.

8.7. Atrybuty WAI-ARIA

Język HTML zapewnia różne funkcje poprawiające dostępność, a jedną z nich jest potężne narzędzie w postaci atrybutów WAI-ARIA (ang. *Web Accessibility Initiative - Accessible Rich Internet Applications*). Atrybuty WAI-ARIA rozszerzają semantykę HTML, dostarczając dodatkowych informacji technologiom wspomagającym, takim jak czytniki ekranu, w celu zwiększenia dostępności stron internetowych.
WAI-ARIA - Overview

WAI-ARIA zapewnia ramy dla dodawania atrybutów do elementów języka HTML w celu identyfikacji funkcji interakcji użytkownika, ich wzajemnych powiązań i bieżącego stanu. WAI-ARIA określa sposoby oznaczania wskazanych obszarów interfejsu użytkownika i współpracujących ze sobą struktur ich elementów składowych (jak np. odnośniki, przyciski, listy, tabele, itp.) jako np. menu, treści podstawowych, treści drugorzędnych, banerów informacyjnych i innych typów struktur. Przykładowo, dzięki WAI-ARIA programiści mogą identyfikować regiony stron i umożliwiać użytkownikom korzystającym wyłącznie z klawiatury łatwe nawigowanie między tymi regionami, bez konieczności wielokrotnego naciskania klawisza Tab.

WAI-ARIA obejmuje również rozwiązania mapujące kontrolki, złożone obszary interfejsów aplikacji i zdarzenia do interfejsów API wykorzystywanych przez narzędzia poprawiające dostępność (np. wspomniane już czytniki ekranowe). Pozwala to nadać znaczenie niestandardowym złożonym kontrolkom (takim jak np. przyciski, lisy rozwijane, widżety kalendarza, rozwijane menu itp.) i w konsekwencji uzyskać zrozumiały sposób nawigowania po takich kontrolkach przez osoby, które np. są niewidome.

WAI-ARIA zapewnia autorom stron i aplikacji internetowych następujące elementy:

- role opisujące typ prezentowanego widżetu, takie jak „menu”, „treeitem”, „slider” i „progressbar”,
- role opisujące strukturę strony internetowej, takie jak nagłówki i regiony,
- właściwości opisujące bieżący stan widżetów, takie jak „zaznaczone” dla pola wyboru lub „tylko do odczytu” dla typowych kontrolek formularzy,
- właściwości definiujące aktywne regiony strony, których zawartość podlegać może aktualizacji w trakcie wyświetlania (np. bieżące notowania giełdowe),
- sposób na zapewnienie nawigacji za pomocą klawiatury dla elementów stron internetowych, takich jak te wymienione powyżej.

Formalny tekst rekomendacji W3C WAI-ARIA (w wersji 1.2 obowiązującej w roku 2024) znajduje się pod adresem:
https://www.w3.org/TR/wai-aria/

Choć jest to dokument opisujący rekomendacje WAI-ARIA bardzo precyzyjnie pod względem formalnym, jego lektura jest dość "ciężka". Z tego względu grupa W3C opublikowała szereg artykułów przedstawiających zalecenia WAI-ARIA w sposób pogrupowany względem zagadnień spotykanych w typowej praktyce twórców aplikacji internetowych. Zostały one wzbogacone o komentarze i przykłady. Zapraszam więc do (dość obszernej) lektury niżej wymienionych artykułów, które poruszają następujące zagadnienia:

- wzorce, czyli przykłady zastosowań zaleceń WAI-ARIA do typowych elementów, z których buduje się strony internetowe (przyciski, odnośniki, menu, slidery, itp.)
  - https://www.w3.org/WAI/ARIA/apg/patterns/
- oznaczenia ról poszczególnych obszarów interfejsu aplikacji, np. nagłówek, stopka, treść główna, treść dodatkowa, nawigacja, podział na sekcje
  - https://www.w3.org/WAI/ARIA/apg/practices/landmark-regions/
- zapewnienie dostępnych nazw i opisów elementów interfejsów aplikacji
  - https://www.w3.org/WAI/ARIA/apg/practices/names-and-descriptions/
- zasady i metody tworzenia funkcjonalności strony internetowej, takie jak menu i siatki, a także interaktywne komponenty, takie jak paski narzędzi i okna dialogowe, obsługiwane za pomocą klawiatury
  - https://www.w3.org/WAI/ARIA/apg/practices/keyboard-interface/
- zasady opisu siatek i tabel za pomocą atrybutów ARIA
  - https://www.w3.org/WAI/ARIA/apg/practices/grid-and-table-properties/
- zasady opisu kontrolek służących do komunikowania wartości numerycznych (np. paski postępu, slidery, scroll-bary, wskaźniki numeryczne itp.)
  - https://www.w3.org/WAI/ARIA/apg/practices/range-related-properties/
- role strukturalne
  - https://www.w3.org/WAI/ARIA/apg/practices/structural-roles/
- ukrywanie semantyki za pomocą roli "prezentacja"
  - https://www.w3.org/WAI/ARIA/apg/practices/hiding-semantics/

Przed rozpoczęciem korzystania z atrybutów WAI-ARIA, należy także zapoznać się z artykułem, który wyjaśnia, dlaczego całkowity brak korzystania z atrybutów WAI-ARIA jest lepszym rozwiązaniem niż wykorzystanie ich w sposób błędny:

- No ARIA is better than Bad ARIA

8.8. Walidatory

Dostępność cyfrową strony można zweryfikować korzystając z dedykowanych ku temu narzędzi. Należy pamiętać, że wskazane niżej programy do sprawdzenia WCAG 2.1 nie mogą być jedyną formą testu - zgodnie z Ustawą niezbędne jest jeszcze sprawdzenie dostępności w sposób „ręczny”.

Przykładowe narzędzia służące do walidacji aplikacji internetowych pod kątem spełniania wymagań dotyczących dostępności cyfrowej:

- WCAG Color Contrast Checker
  - Chrome Web Store / WCAG Color Contrast Checker
  - Narzędzie dostępne jest w formie wtyczki do przeglądarki Chrome i służy (zgodnie ze swoją nazwą) do sprawdzenia kontrastu pomiędzy poszczególnymi elementami wizualnymi strony internetowej. Obsługa polega na uruchomieniu rozszerzenia (po kliknięciu w przycisk), po czym z prawej strony zostanie otwarta lista par kolorystycznych na wybranej podstronie. U góry należy określić, dla jakiej oceny przeprowadzamy przegląd: A, AA czy AAA.
- WAVE Web Accessibility Evaluation Tools
  - https://wave.webaim.org/
  - Walidator jest narzędziem on-line. Po wpisaniu adresu strony przeprowadza jej analizę pod kątem spełnienia wymagań w zakresie dostępności. Badaną stronę można również sprawdzić pod kątem poprawności struktury i układu elementów na stronie wyłączając style CSS (poprzez kliknięcie w zakładkę "No styles"). Ostatnią użyteczną zakładką jest "Contrast" - po jej kliknięciu analizowany jest kontrast na stronie oraz zastosowane kolory tekstu i tła. Stosując powyższe narzędzie można dokonać audytu strony pod kątem kilku podstawowych wytycznych WCAG 2.0, ale tylko tych, które można automatycznie sprawdzić analizując kod strony.
  - Narzędzie zostało opracowane i udostępnione nieodpłatnie dzięki Utah State University.
  - Omówienie możliwości tego walidatora w formie krótkiego filmu:



- Accessibility Checker
  - https://www.accessibilitychecker.org/
  - Walidator online. Pozwala wybrać kraj, względem którego regulacji prawnych odbywa się testowanie.
- W3C Markup Validation Service
  - https://validator.w3.org/
  - Walidator jest narzędziem on-line i służy do sprawdzania poprawności zastosowanego kodu HTML badanej strony ze standardami HTML i XHTML. Po wyświetleniu strony startowej należy wpisać adres www weryfikowanej witryny, następnie po kliknięciu przycisku „Check” wyświetlany jest szczegółowy raport wskazujący wykryte na stronie błędy i nieprawidłowości w kodzie HTML.
- Squidler
  - https://squidler.io/
  - Wiele narzędzi do skanowania i testowania stron internetowych automatycznie bierze pod uwagę tylko odpowiedzi HTTP - kody stanu, nagłówki i renderowany po stronie serwera kod HTML. Jednak w przypadku wielu nowoczesnych witryn i aplikacji korzystających intensywnie z JavaScriptu do dynamicznego budowania interfejsu użytkownika, takie narzędzia mają problem z oceną treści renderowanych po załadowaniu strony. Co więcej, wszystko, co wymaga interakcji użytkownika, aby zostać wyrenderowane, jak np. otwarcie okna modalnego, jest poza zasięgiem prostych walidatorów kodu.
    Squidler używa prawdziwej przeglądarki do renderowania i interakcji ze stronami internetowymi i aplikacjami. Nie ma znaczenia, jak strona jest skonstruowana, o ile jest renderowana w DOM. Statyczne strony internetowe, renderowanie po stronie serwera,

progresywne aplikacje internetowe i aplikacje jednostronicowe - wszystkie mogą być testowane.

Obszerną listę narzędzi tego typu można znaleźć na stronie organizacji W3C Web Accessibility Evaluation Tools List.

8.9. Przykładowe wdrożenia

Prezentacja **Demo Przed i Po** jest wielostronicowym zasobem zawierającym niedostępną witrynę internetową i poprawioną wersję tej samej witryny. Każda strona internetowa zawiera adnotacje śródliniowe, które można uaktywnić, aby wyróżnić niektóre najważniejsze bariery dostępności lub poprawki. Do każdej strony internetowej dołączony jest raport oceniający, który informuje projektantów stron o zgodności z wytycznymi WCAG.

Prezentacja jest dostępna na stronie Pracowni Dostępności Cyfrowej LepszyWeb.pl.

Z kolei na stronie Smultron Web Development można zapoznać się z artykułem opisującym szczegóły techniczne wdrożenia WCAG 2.1 na przykładzie projektu uczelniadostepna.pl.